INTERVIEW STUDY GUIDE · VOLUME 1

AM13E230 TINIElite Edge AI Project from First Principles

What the project is, what problem it solves, what happens from input to output, and exactly what was built.

Author: Jainil Bavishi
Platform: TI LP-AM13E230 LaunchPad · Arm Cortex-M33 · TINIElite NPU · Zephyr RTOS
Documentation baseline: 23 October 2026

Contents

1. The project in plain English
2. The engineering problem
3. Input, processing, and output
4. Concepts needed to understand the system
5. Complete system architecture
6. End-to-end execution flow
7. What we implemented and why
8. Hardware and memory map
9. Repository organization
10. Ownership, proof, and project boundaries
11. How to explain it aloud

1. The project in plain English

> **ONE-SENTENCE DEFINITION**
> This project turns a TI LP-AM13E230 microcontroller board into a reliable, testable edge-AI waveform classifier that runs a generated neural network partly on the chip's TINIElite NPU under Zephyr RTOS.

The input represents one waveform as **256 numerical samples**. The host computer converts those floating-point samples into the signed 8-bit format expected by the model and sends them to the board over USB UART. The board runs inference and returns three signed scores in this fixed order:

Output index	Class	Meaning
0	Sawtooth	Model score for a sawtooth waveform
1	Sine	Model score for a sine waveform
2	Square	Model score for a square waveform

The largest score is treated as the predicted class. These values are signed `int8` model outputs, not calibrated probabilities. The board also returns measured execution times and NPU interrupt counters so the host can verify that hardware acceleration actually occurred.

Edge AI means the inference is executed on the device at the edge of the system rather than sending the tensor to a cloud server. The PC currently supplies test data and checks results, but it does not perform the neural-network inference.

2. The engineering problem

The available starting point was not a complete application. The important supplied artifact was a TI/TVM-generated static library named `mod.a`, plus its generated C header and six test fixtures. The library contained compiled neural-network code, but a usable embedded system still needed all of the surrounding infrastructure.

What was available

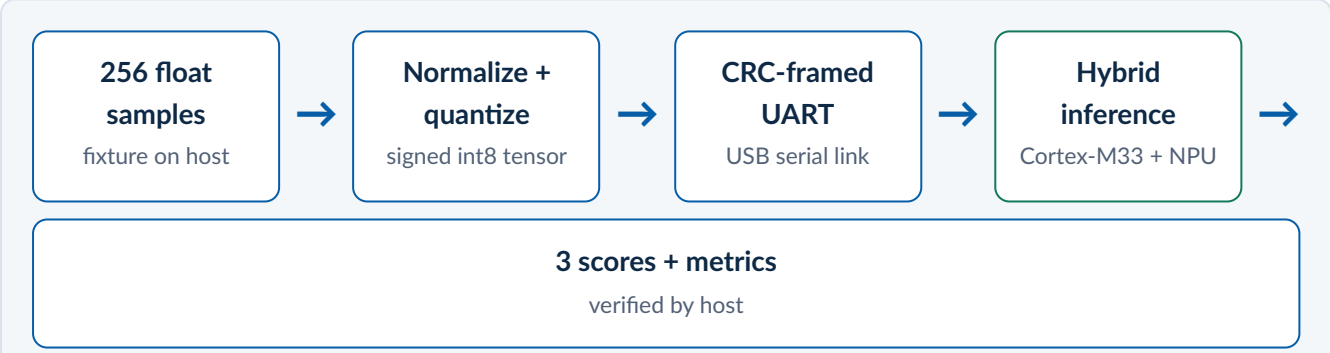
- LP-AM13E230 development board.
- Arm Cortex-M33 and TINIElite NPU hardware.
- Generated model archive `mod.a`.
- Generated API header and quantization constants.
- Six golden regression fixtures.
- TI technical collateral and debug tools.

What was missing

- Zephyr support for this preview board and SoC.
- UART and pin-control drivers.
- NPU power, memory, and interrupt integration.
- A robust host-to-board protocol.
- Timeout and deadlock recovery.
- Reproducible WSL build, flash, test, and CI workflows.

The real project was therefore broader than "run a model." It was a **platform bring-up, device-driver, runtime-integration, communications, reliability, and validation project** centered on a real edge-AI workload.

3. Input, processing, and output



3.1 Model shape

- Generated input tensor: `(1, 1, 256, 1)`, signed 8-bit.
- Generated output tensor: `(1, 3)`, signed 8-bit.
- The dimensions of size one are batch/channel layout details. The meaningful sample dimension is 256.

3.2 Quantization

Embedded accelerators commonly use integer arithmetic because it needs less memory and is efficient in hardware. The generated header defines this conversion:

```
input_int = clip(int((input_float + bias) × scale) >> shift, -128, 127)

bias  = -70.02075
scale = 187.0
shift = 6
```

The Python client reads these constants directly from the generated header instead of duplicating them manually. It applies C/Python integer truncation, performs the right shift, and clips the result to the valid `int8` range. Matching preprocessing matters: even a correct model can give different outputs if the host uses a different scale, rounding rule, order, or data type.

3.3 What the output proves

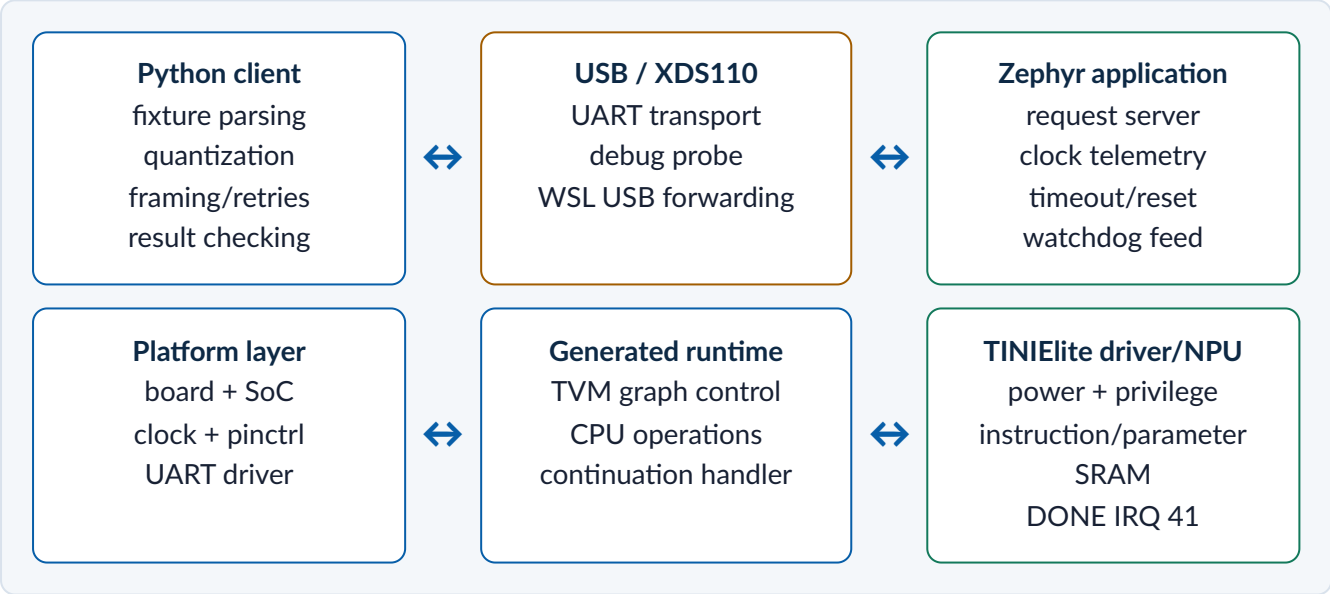
The host compares all three returned bytes with the corresponding golden output. An exact match on the six fixtures proves that this firmware revision reproduces those six supplied references. It does **not** establish general accuracy on unseen data.

4. Concepts needed to understand the system

Concept	Simple explanation	How it appears here
Microcontroller (MCU)	A small computer on one chip with CPU, memory, and peripherals.	AM13E23019 with an Arm Cortex-M33.
RTOS	An operating system for predictable embedded tasks, drivers, timing, and synchronization.	Zephyr supplies startup, device model, kernel primitives, tests, and build tooling.
NPU	Hardware specialized for neural-network operations.	TINIElite executes generated convolution/pooling stages.
Driver	Software that converts a standard or project API into hardware register operations.	Custom UART, pinctrl, and TINIElite integration.
MMIO	Peripherals controlled by reading/writing fixed memory addresses.	`sys_read32` and `sys_write32` access clock, UART, watchdog, and NPU registers.
Interrupt	A hardware event that temporarily transfers control to an interrupt handler.	NPU DONE on IRQ 41 wakes the waiting application.
Semaphore	A kernel synchronization counter used to block until an event occurs.	The ISR gives; the inference thread takes with a 100 ms timeout.
UART	An asynchronous serial byte stream.	115200 baud communication between host and board.
CRC-32	A strong error-detecting code for accidental corruption.	Every protocol frame protects header fields and payload.

Watchdog	An independent timer that resets a stuck system if software stops showing progress.	WWDT0 resets after four seconds without a feed.
Devicetree	Structured hardware description consumed at build time.	Defines addresses, clocks, interrupts, memory, and enabled devices.
Kconfig	Compile-time feature configuration system.	Enables drivers and gates dangerous fault-injection commands.
ABI	Binary-level contract between separately compiled code.	`mod.a` expects specific NPU memory functions and exports runtime functions.
Static library	Compiled object code linked into the final executable.	`mod.a` is linked into `zephyr.elf`.

5. Complete system architecture



5.1 Why Zephyr?

Zephyr provides a standard device model, build system, compile-time hardware description, kernel synchronization, time conversion, reboot services, native simulation, and a large testing ecosystem. The alternative would be to write a completely bare-metal startup and scheduling environment. The project still requires low-level register work because AM13E230 support was unavailable in the pinned upstream revision.

5.2 What “out-of-tree” means

The board and driver support live in this repository rather than being copied into the upstream Zephyr checkout. Zephyr discovers the repository as a module through `zephyr/module.yml`. This prevents local patches from being mixed into millions of upstream lines, makes Git history focused, and allows a clean, pinned Zephyr checkout to reproduce the build.

5.3 Why the inference is hybrid

The generated runtime owns the graph. It performs orchestration and some operations on the Cortex-M33, while generated TINIElite kernels offload supported stages. The current ELF contains three convolution entry points and one average-pooling entry point, plus CPU-side generated functions. Five acknowledged NPU completions are observed per inference. Because the graph and sublayer schedule are contained in a binary archive, the documentation does not invent a layer-by-layer network architecture that cannot be independently inspected.

6. End-to-end execution flow

1. **Power-on:** the Cortex-M33 begins Zephyr startup.
2. **Clock initialization:** a pre-kernel routine safely configures flash wait states, starts the 25 MHz crystal, locks the PLL, and moves MCLK to 200 MHz.
3. **Driver initialization:** Zephyr initializes pin routing, UNICOMM UART, TINIElite power/privilege/interrupt state, and application devices.
4. **Safety initialization:** the application configures WWDT0 for an independent four-second reset and verifies the NPU device is ready.
5. **Readiness:** the firmware drains stale UART bytes and sends a CRC-protected `READY` frame declaring 256 input bytes and three outputs.
6. **Host preprocessing:** Python reads a float fixture and quantizes 256 values to `int8`.
7. **Request:** Python sends a protocol-v2 frame containing type, sequence, payload length, tensor, and CRC-32.
8. **Validation:** firmware checks UART errors, magic, version, type, length, deadline, and CRC.
9. **Acknowledgement:** a valid inference request receives a sequence-matched `ACK`.
10. **Runtime start:** the application prepares interrupt state and calls `tvmgen_default_run()`.
11. **NPU execution:** generated code copies microcode/parameters into the TINIElite SRAM windows and starts a hardware stage.
12. **Completion:** DONE IRQ 41 enters the small driver ISR, which acknowledges the interrupt, increments a counter, and gives a semaphore.
13. **Continuation:** the application wakes and calls the generated `TI_NPU_NNC_IRQHandler()` in thread context. The runtime either launches another stage or marks the graph finished.
14. **Response:** firmware returns three scores, accelerator wait time, runtime time, total time, completion count, spurious count, and CRC.
15. **Verification:** the host checks frame integrity, sequence, exact scores, nonzero completions, zero spurious IRQs, and optional stress/fault-test criteria.

7. What we implemented and why

Implemented area	Why it was necessary	Key result

Out-of-tree board/SoC port	The preview MCU was not supported by the pinned Zephyr tree.	Zephyr can build for `lp_am13e230/am13e23019`.
Clock initialization and telemetry	Correct timing, UART baud, flash safety, and latency claims require a known clock.	Read-back validated 200 MHz MCLK and three flash wait states.
Pinctrl integration	PA0/PA1 must be routed to UNICOMM UART functions.	Standard Zephyr pinctrl data replaces hardcoded application pin writes.
Polling UART driver	No suitable Zephyr driver existed for the dual-base UNICOMM peripheral.	TX, RX, baud configuration, and hardware error reporting work.
Protocol v2	A raw fixed packet/checksum could not reliably detect, correlate, or recover from failures.	Length, version, sequence, CRC-32, errors, deadlines, resynchronization, retries.
TINIElite adapter	The generated archive expects hardware memory callbacks and completion handling.	Power/reset/privilege, bounded SRAM writes, IRQ 41, semaphore delivery.
Hybrid runtime integration	Generated execution continues in stages rather than one blocking NPU call.	Runtime start/wait/continuation loop with metrics.
NPU timeout recovery	A missing interrupt must not hang forever or leave opaque runtime state reused.	100 ms bounded wait, structured error, cold reset, restored readiness.
Independent WWDT0 watchdog	A software timeout cannot recover code that is completely deadlocked.	Four-second hardware reset, reset-cause read-back, validated restart.
Python regression client	Repeatable preprocessing, transport, scoring, metrics, and fault tests were needed.	One client supports fixtures, stress, clocks, CRC, NPU stall, and watchdog tests.
WSL-native workflow	The complete developer flow needed to work from Linux under WSL.	Build, Linux UniFlash/DSLite flash, UART test, and USB permission scripts.
Automated quality gates	A hardware demo needs reproducible software evidence.	Host tests, sanitized ztests, lint/format/static checks, cross-build, symbol checks, artifacts.

8. Hardware and memory map

Resource	Address / configuration	Use
Flash	`0x00000000`, 512 KiB	Firmware and linked model archive.
Main SRAM	`0x20000000`, 96 KiB	Stacks, globals, input/output, runtime working memory.
Separate SRAM/code alias	`0x00c18000`, 32 KiB	Described by SoC devicetree; not the primary linked SRAM.

Core	Arm Cortex-M33, MCLK 200 MHz	Zephyr, runtime orchestration, CPU graph stages.
UART	`0x40601000`, 115200 baud	Data/status registers and host link.
UNICOMM wrapper	`0x40632000`	Power, reset, and peripheral mode.
TINIElite	`0x408ff000`, IRQ 41	Neural-network acceleration and interrupt registers.
NPU instruction SRAM	NPU base + `0x2000`, 256 words	Generated microcode staging.
NPU parameter SRAM	NPU base + `0x2800`, 512 words	Per-stage parameter staging.
WWDT0	`0x400d0000`, 32.768 kHz source	Independent four-second liveness supervision.
XDS110	USB VID:PID `0451:bef3`	Debug probe and application UART bridge.

9. Repository organization

Path	Responsibility
`app/`	Main server, clock telemetry, NPU orchestration, protocol, UART framing, watchdog.
`boards/`	LaunchPad identity, CPU clock, enabled peripherals, and UART pins.
`soc/`	AM13x Kconfig identity, CPU features, early clock setup, pinctrl data types.
`dts/`	Memory/peripheral description and binding schemas.
`drivers/`	TINIElite adapter, UNICOMM UART, and pin controller.
`include/`	Public project driver API and pin constants.
`models/`	Binary model archive, generated header, license, hashes, and six fixtures.
`tools/`	Python host/regression client.
`scripts/`	Build, flash, test, pipeline, static checks, and XDS110 setup.
`tests/`	Python unit tests and Zephyr `native_sim` protocol tests.
`docs/`	Architecture, protocol, audit, validation, evidence, and roadmap.
`west.yml`	Pins Zephyr and the required CMSIS modules.
`.github/workflows/ci.yml`	Public software quality and firmware build automation.

10. Ownership, proof, and project boundaries

Implemented and validated

- Zephyr platform integration.
- Clock, pinctrl, UART, and NPU glue.
- Interrupt completion and bounded recovery.
- Watchdog supervision.
- Framed protocol and host client.
- WSL-native build/flash/test.
- Automated software and hardware evidence.

Provided externally

- Trained neural-network model.
- Generated `mod.a` and header.
- Six golden vectors.
- TI NPU runtime internals inside the archive.
- Hardware documentation and UniFlash tooling.

> **DO NOT OVERCLAIM**
> Six fixtures are a regression set, not an accuracy dataset. There is no source model, training code, independent labeled dataset, CPU-equivalent artifact, or power measurement. The system is a strong hardware-validated engineering prototype, not a production-certified product.

10.1 What proves real NPU use?

- The linked ELF contains generated TINIElite convolution and pooling symbols.
- The generated archive calls project-supplied NPU instruction/parameter memory callbacks.
- Physical IRQ 41 produces exactly five acknowledged completions per inference.
- The host rejects a result with zero completions or any spurious completion.
- A deliberate missing-completion test reaches the real semaphore timeout path and resets cleanly.

Timing alone is not treated as proof. Direct symbol, MMIO, interrupt, and fault-path evidence is stronger.

11. How to explain it aloud

Thirty-second version

> I built the platform and reliability layers needed to run a TVM-generated waveform classifier on TI's AM13E230 microcontroller under Zephyr. A Python client quantizes 256 samples and sends them through a CRC-protected UART protocol. The firmware runs a hybrid CPU/NPU graph, waits for real TINIElite completion interrupts, and returns three class scores plus timing and interrupt evidence. I also added clock validation, NPU timeout recovery, an independent hardware watchdog, WSL-native flashing, tests, and CI. I integrated the supplied binary model; I did not train it.

Two-minute version

> The target was a preview TI AM13E230 LaunchPad with a Cortex-M33 and a TINIElite neural accelerator. The only model-side inputs were a generated static library, its header, and six reference vectors, while the pinned Zephyr version did not support the board. I created an out-of-tree board and SoC port, configured the real 200 MHz clock tree and flash wait states, and implemented pinctrl, polling UART, and the NPU adapter expected by the generated binary. On the host, Python reproduces the model's int8 preprocessing and exchanges versioned, sequence-numbered, CRC-32 frames with the firmware. During inference, the generated runtime alternates between CPU orchestration and NPU stages. IRQ 41 is acknowledged in a small ISR, which signals a semaphore; the application continues the generated runtime in thread context with a 100 ms bound. A timeout produces a structured error and cold reset because the binary runtime may retain opaque state. A separate four-second hardware watchdog covers total firmware deadlock. The complete flow builds, flashes, and tests natively in WSL. On hardware, six fixtures passed exactly, and a 600-inference run produced 3,000 acknowledged NPU interrupts, zero spurious interrupts, and zero transport retries. Those are regression results, not a claim of general model accuracy.

Continue with Volume 2 for the source-level implementation and register-by-register reasoning.